

## Exercise 07: *PyUVM* Universal Verification Components integration

**Objective:** Introduction to reusable *PyUVM* Universal Verification Components.

**Task:** Updating the existing *Saturation Filter* testbench to drive and observe its *sSDT* interfaces with producer and consumer *UVC* agents.

In this exercise you will update your solution from *Exercise 06: PyUVM Environment, Sequencer, and Sequences* to use the provided *sSDT UVC*. Start from your completed Exercise 06 testbench and copy it to `<ROOT>/lecture04/exercises/E07_sat_pyuvvm.uvc`.

See `<ROOT>/lectures/common/docs/exercise_general_info.pdf` for more information on the *sSDT UVC*.

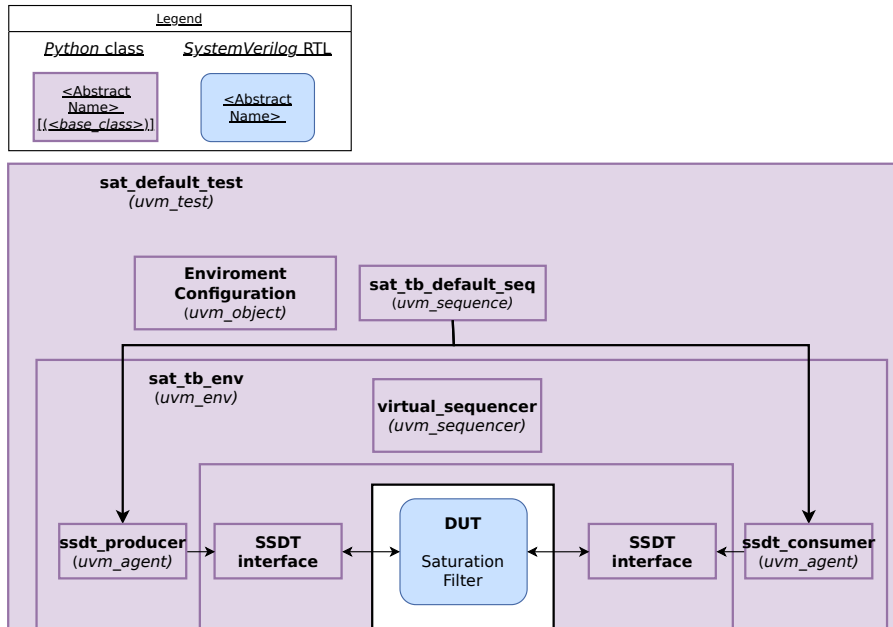


Fig. 0.1: Testbench overview. This exercise implements adds the producer and consumer *UVC*.

The *Saturation Filter* testbench needs two instances of the agent:

- A *producer agent* drives the DUT input *sSDT* interface.
- A *consumer agent* waits for transactions on the DUT output *sSDT* interface.

The top virtual sequence no longer drives or samples interface signals directly. Instead, it starts *sSDT* transactions in the form of sequences on the sequencers contained in the two agents.

The required *sSDT UVC* classes can be imported from `uvc.ssdt` directly. `uvc/ssdt/__init__.py` exports all available classes and functions from the individual source files.

### Part 1: Testbench Configuration

Update the testbench configuration in `sat_tb_config.py`. Import `uvc_ssdt_config` from `uvc.ssdt` and create two configuration objects using the factory:

- `ssdt_prod_cfg`, to configure the producer agent.
- `ssdt_cons_cfg`, to configure the consumer agent.

These configurations replace the input and output interface handles previously stored in the configuration object. Each `uvc_ssdt_config` provides:

- `vif`; for the interface handle
- `is_active`; to set the agent to active or passive
- `driver_type`; to set the agent to producer or consumer

The interface handle is going to be set from the test, the agent configuration should be done in the `__init__` configuration object.

- In the producer configuration set:
  - `is_active` to `uvm_active_passive_enum.UVM_ACTIVE`
  - `driver_type` to `uvc_ssdt_type_enum.PRODUCER`.
- In the consumer configuration set:
  - `is_active` to `uvm_active_passive_enum.UVM_ACTIVE`
  - `driver_type` to `uvc_ssdt_type_enum.CONSUMER`.

In `sat_tb_default_test.py` replace the interface wrapper class with the provided `ssdt_interface_wrapper` from `uvc.ssdt`. This is the exact same class, we're simply importing it from the *UVC* instead of the interface class source file from Exercise 06.

Assign the handles to `cfg.ssdt_prod_cfg.vif` and `cfg.ssdt_cons_cfg.vif`, respectively. Keep the `ConfigDB set` call that passes the configuration object to the environment.

Also update the `connect` phase if any variables have changed.

## Part 2: Producer and Consumer Agents

Update `sat_tb_env.py` to instantiate the two *sSDT UVC* agents.

Add empty `uvc_ssdt_producer` and `uvc_ssdt_consumer` handles to the environments `__init__`. In the `build_phase`, after retrieving the testbench configuration and creating the virtual sequencer:

- Create `uvc_ssdt_producer` using the factory.
- Store `cfg.ssdt_prod_cfg` in the `ConfigDB` for the producer agent using the field name `cfg`.

---

```
ConfigDB().set(
    context    = self,
    inst_name  = "uvc_ssdt_producer",
    field_name = "cfg",
    value      = self.cfg.ssdt_prod_cfg,
)
```

---

- Create `uvc_ssdt_consumer` using the factory.
- Store `cfg.ssdt_cons_cfg` in the ConfigDB for the consumer agent using the field name `cfg`.

## Part 3: Virtual Sequencer Connections

The top-level virtual sequence needs access to the sequencer inside each agent. In `sat_tb_virt_sequencer.py`, add two empty handles to the `__init__`:

- `ssdt_producer_sequencer`; for the producer agent's sequencer.
- `ssdt_consumer_sequencer`; for the consumer agent's sequencer.

Keep the configuration handle and its ConfigDB retrieval.

Add a `connect_phase` to `sat_tb_env.py`. Connect the agent sequencers to the corresponding virtual sequencer handles:

- Assign `uvc_ssdt_producer.sequencer` to `virtual_sequencer.ssdt_producer_sequencer`.

---

```
self.virtual_sequencer.ssdt_producer_sequencer = self.uvc_ssdt_producer.sequencer
```

---

- Assign `uvc_ssdt_consumer.sequencer` to `virtual_sequencer.ssdt_consumer_sequencer`.

## Part 4: Transaction-Level Virtual Sequence

Update the default sequence in `sat_tb_default_seq.py`. Instead of driving the interfaces directly we now drive sequences on the *sSDT UVC* sequencers

Import `uvc_ssdt_basic_seq` and `uvc_ssdt_seq_item` from `uvc.ssdt`.

`uvc_ssdt_basic_seq` drives a single sequence item on the driver. The sequence item needs to be created in the virtual sequence and passed to the producer sequence. The consumer sequence needs no sequence item.

In the sequence constructor using the factory create:

- A producer `uvc_ssdt_basic_seq`.
- A consumer `uvc_ssdt_basic_seq`.

For each transaction in the sequence body:

1. Create a `uvc_ssdt_seq_item` using the factory.
2. Assign the input value to its `data` field.
3. Assign the item to the producer sequence's `seq_item` field.

---

```
seq_item = uvc_ssdt_seq_item.create("producer_item")
seq_item.data = 46
self.producer_seq.seq_item = seq_item
```

---

4. Start the producer sequence on `self.sequencer.ssdt_producer_sequencer`.

5. Start the consumer sequence on `self.sequencer.ssdt_consumer_sequencer`.
6. Run both sequences concurrently using `cocotb.start_soon()` and wait for both tasks using `Combine`.

---

```
prod_task = cocotb.start_soon(  
    self.producer_seq.start(  
        seqr=self.sequencer.ssdt_producer_sequencer  
    )  
)  
cons_task = cocotb.start_soon(  
    self.consumer_seq.start(  
        self.sequencer.ssdt_consumer_sequencer  
    )  
)  
  
# Finishes when the both tasks finishes  
await Combine(prod_task, cons_task)
```

---

Send two transactions, one that saturates the *Saturation Filter* and one that does not saturate. (Hint: You can reuse the sequences, just assign a new sequence item.)

## Part 5: Running the Test

Run the test using the `Makefile` and verify that both the below-threshold and above-threshold transactions pass.